hello-lucky · personal account 699558695655 · us-east-1 · profile personal

Deploying hello-lucky

A Spring Boot service on Java 21. `GET /hello-lucky/hello?country=US` returns the country, a phone country code, and a lucky number.

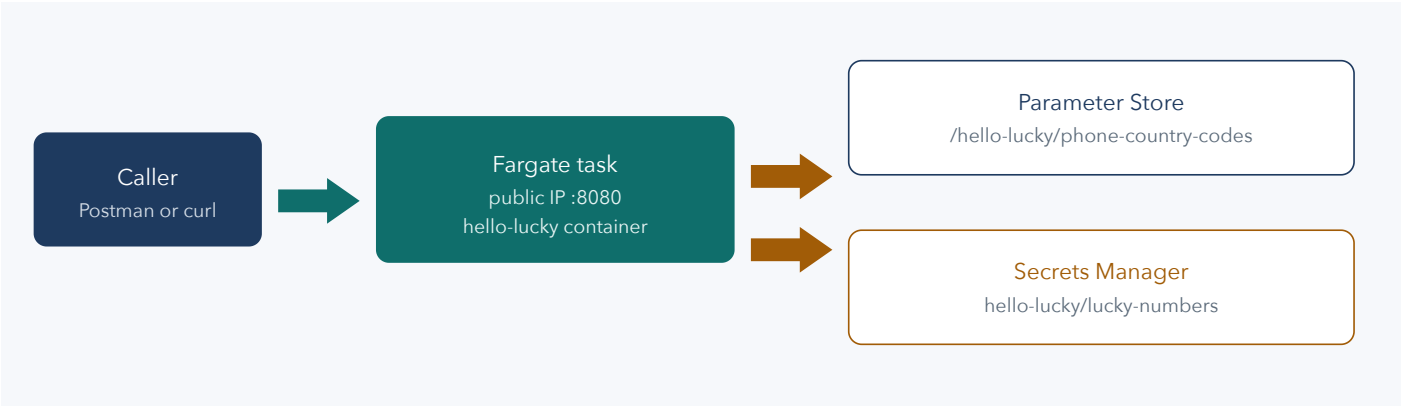


Figure 2. Each request reads both maps. US returns phone code 1 and lucky number 11. INDIA returns 91 and 22.

Build, test, and the container image

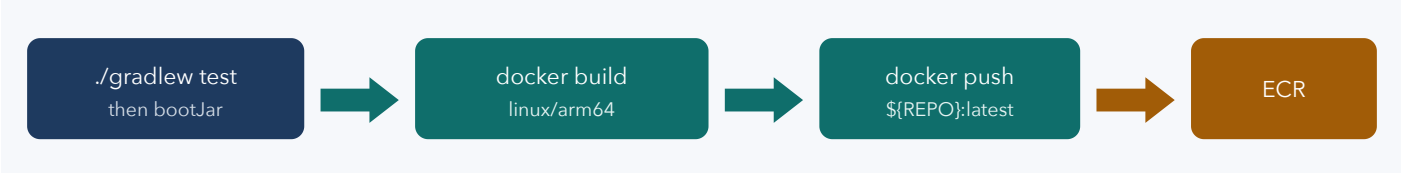


Figure 1. The jar is built first. The image is pushed before ECS starts a task.

These commands are the same for the AWS CLI path and the Terraform path. Run them from the project root, `/Users/snagulapalli/try/hello-lucky`.

```
./gradlew test
./gradlew bootJar
```

`bootJar` is registered by the Spring Boot plugin in `build.gradle`. It writes `build/libs/hello-lucky-0.0.1-SNAPSHOT.jar`. It does not look at the Dockerfile.

Run it on the laptop, with the personal profile and an explicit region. The Java SDK does not read a config heading written as `[personal]`. It expects `[profile personal]`.

```
AWS_PROFILE=personal AWS_REGION=us-east-1 ./gradlew bootRun
```

Then call `http://localhost:8080/hello-lucky/hello?country=US`.

The image copies that jar onto Amazon Corretto 21:

```
docker build --platform linux/arm64 -t hello-lucky:latest .
```

This Mac builds an ARM image, so the ECS task must use `ARM64`. `EXPOSE 8080` only records that the process listens on 8080. The security group and the task port mapping are what let a caller reach it.

1. Run using the AWS CLI

This is the manual deploy. Each command creates one piece, and the shell variables carry ids into the next command. Set the profile and region first, and keep the shell open so `$VPC_ID` and the other variables stay set.

```
export AWS_PROFILE=personal
export AWS_REGION=us-east-1
export AWS_DEFAULT_REGION=us-east-1
ACCOUNT=$(aws sts get-caller-identity --query Account --output text)
```

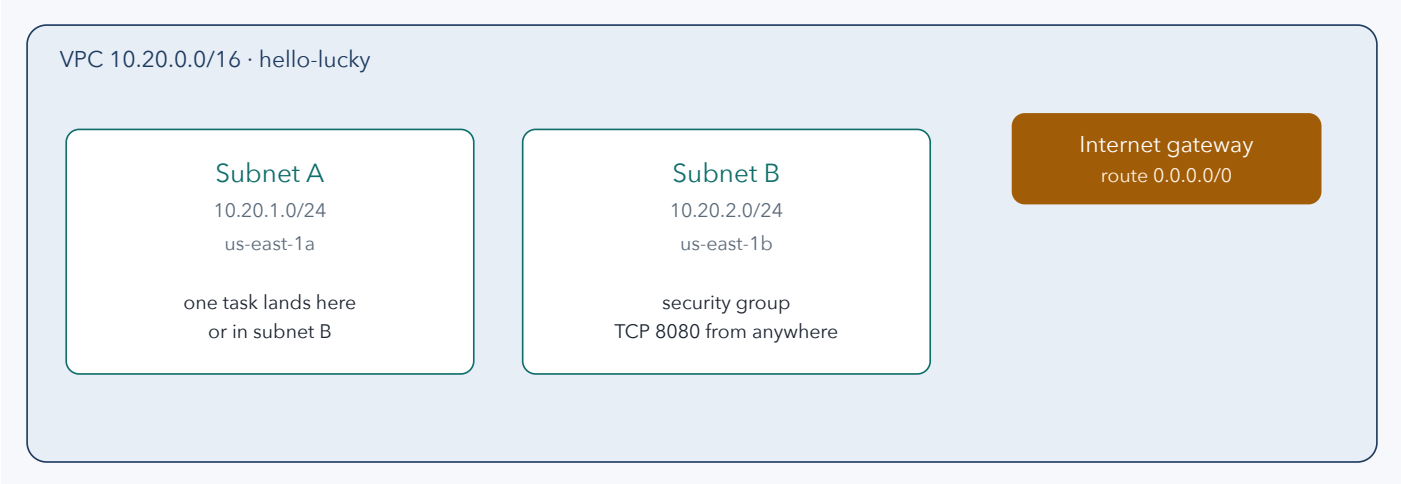


Figure 3. Both subnets share one route table. A single task is placed in only one of them.

1. **VPC.** `10.20.0.0/16` is the address range. DNS support lets tasks resolve names such as the ECR hostname. DNS hostnames lets a public IP also have a public DNS name. Hostnames work only when support is also on.

```
VPC_ID=$(aws ec2 create-vpc --cidr-block 10.20.0.0/16 \
  --tag-specifications 'ResourceType=vpc,Tags=[{Key=Name,Value=hello-lucky}]' \
  --query 'Vpc.VpcId' --output text)
aws ec2 modify-vpc-attribute --vpc-id "$VPC_ID" --enable-dns-support '{"Value":true}'
aws ec2 modify-vpc-attribute --vpc-id "$VPC_ID" --enable-dns-hostnames
'{"Value":true}'
```

2. **Internet gateway.** Create it, then attach it to the VPC. In zsh, type plain hyphens and straight quotes. An en dash makes the CLI report that `–-vpc-id` is missing.

```
IGW_ID=$(aws ec2 create-internet-gateway \
  --tag-specifications 'ResourceType=internet-gateway,Tags=[{Key=Name,Value=hello-
lucky}]' \
  --query 'InternetGateway.InternetGatewayId' --output text)
aws ec2 attach-internet-gateway --internet-gateway-id "$IGW_ID" --vpc-id "$VPC_ID"
```

3. **Two subnets** inside the VPC, in two availability zones.

```
SUBNET_A=$(aws ec2 create-subnet --vpc-id "$VPC_ID" --cidr-block 10.20.1.0/24 --
availability-zone us-east-1a \
  --tag-specifications 'ResourceType=subnet,Tags=[{Key=Name,Value=hello-lucky-a}]' \
  --query 'Subnet.SubnetId' --output text)
SUBNET_B=$(aws ec2 create-subnet --vpc-id "$VPC_ID" --cidr-block 10.20.2.0/24 --
availability-zone us-east-1b \
  --tag-specifications 'ResourceType=subnet,Tags=[{Key=Name,Value=hello-lucky-b}]' \
  --query 'Subnet.SubnetId' --output text)
```

4. **Route table.** Creating the table also installs an implicit route for `10.20.0.0/16` to `local`. The extra route sends every other IPv4 address to the gateway. Each association is its own API call. A subnet with no association keeps using the VPC's main route table.

```
RTB_ID=$(aws ec2 create-route-table --vpc-id "$VPC_ID" \
  --tag-specifications 'ResourceType=route-table,Tags=[{Key=Name,Value=hello-lucky}]'
\
  --query 'RouteTable.RouteTableId' --output text)
aws ec2 create-route --route-table-id "$RTB_ID" --destination-cidr-block 0.0.0.0/0 --
gateway-id "$IGW_ID"
aws ec2 associate-route-table --route-table-id "$RTB_ID" --subnet-id "$SUBNET_A"
aws ec2 associate-route-table --route-table-id "$RTB_ID" --subnet-id "$SUBNET_B"
```

5. **Security group.** A new group allows all outbound traffic and no inbound traffic. The second command opens TCP 8080 from any IPv4 address. `0.0.0.0/0` is the source address, not the port.

```
SG_ID=$(aws ec2 create-security-group --group-name hello-lucky \
  --description "hello-lucky HTTP 8080" --vpc-id "$VPC_ID" --query 'GroupId' --output
text)
aws ec2 authorize-security-group-ingress --group-id "$SG_ID" --protocol tcp --port
8080 --cidr 0.0.0.0/0
```

6. **Execution role.** ECS may assume `hello-lucky-execution`. The attached AWS-managed policy lets Fargate pull from ECR and write logs. The Java process does not use this role.

```
aws iam create-role --role-name hello-lucky-execution \
  --assume-role-policy-document '{"Version":"2012-10-17","Statement":
[{"Effect":"Allow","Principal":{"Service":"ecs-
tasks.amazonaws.com"},"Action":"sts:AssumeRole"}]}'
aws iam attach-role-policy --role-name hello-lucky-execution \
  --policy-arn arn:aws:iam::aws:policy/service-role/AmazonECSTaskExecutionRolePolicy
```

7. **Log group,** kept for 7 days.

```
aws logs create-log-group --log-group-name /ecs/hello-lucky
aws logs put-retention-policy --log-group-name /ecs/hello-lucky --retention-in-days 7
```

8. **ECR repository, then push.** Login stores a temporary password for that registry host. The image name is what tells `docker push` where to send the layers. In zsh, write `${REPO}:latest`. A bare `$REPO:latest` is mangled.

```
aws ecr create-repository --repository-name hello-lucky
REPO="${ACCOUNT}.dkr.ecr.us-east-1.amazonaws.com/hello-lucky"
aws ecr get-login-password --region us-east-1 | docker login --username AWS --
password-stdin "${ACCOUNT}.dkr.ecr.us-east-1.amazonaws.com"
docker tag hello-lucky:latest "${REPO}:latest"
docker push "${REPO}:latest"
```

9. **Cluster.** An empty group. It does not start a container.

```
aws ecs create-cluster --cluster-name hello-lucky
```

0. **Task definition.** Save the JSON below as `task-definition.json` and register it. The family name stays `hello-lucky`. The first registration is revision 1. Registering again after a change creates revision 2. This copy is the manual blueprint: 0.5 vCPU, 1 GB, ARM64, port 8080.

```
aws ecs register-task-definition --cli-input-json file://task-definition.json
```

```
{
  "family": "hello-lucky",
  "networkMode": "awsvpc",
  "requiresCompatibilities": ["FARGATE"],
  "cpu": "512",
  "memory": "1024",
  "runtimePlatform": {
    "cpuArchitecture": "ARM64",
    "operatingSystemFamily": "LINUX"
  },
  "executionRoleArn": "arn:aws:iam::699558695655:role/hello-lucky-execution",
  "containerDefinitions": [{
    "name": "hello-lucky",
    "image": "699558695655.dkr.ecr.us-east-1.amazonaws.com/hello-lucky:latest",
    "essential": true,
    "portMappings": [{ "containerPort": 8080, "protocol": "tcp" }],
    "logConfiguration": {
      "logDriver": "awslogs",
      "options": {
        "awslogs-group": "/ecs/hello-lucky",
        "awslogs-region": "us-east-1",
        "awslogs-stream-prefix": "ecs"
      }
    }
  }]
}
```

1. **Service.** Desired count 1 starts one task and keeps it running. ECS chooses one of the two subnets. `assignPublicIp=ENABLED` gives that task a public address.

```
aws ecs create-service \
  --cluster hello-lucky \
  --service-name hello-lucky \
  --task-definition hello-lucky \
  --desired-count 1 \
  --launch-type FARGATE \
  --network-configuration "awsvpcConfiguration={subnets=
[$SUBNET_A,$SUBNET_B],securityGroups=[$SG_ID],assignPublicIp=ENABLED}"
aws ecs wait services-stable --cluster hello-lucky --services hello-lucky
```

2. **Public IP and a call.** `aws ecs wait` does not print an address. These three commands do. The app needs about 15 seconds after the task is running before Tomcat listens.

```
TASK=$(aws ecs list-tasks --cluster hello-lucky --service-name hello-lucky --desired-
status RUNNING --query 'taskArns[0]' --output text)
ENI=$(aws ecs describe-tasks --cluster hello-lucky --tasks "$TASK" --query
"tasks[0].attachments[0].details[?name=='networkInterfaceId'].value | [0]" --output
text)
PUBLIC_IP=$(aws ec2 describe-network-interfaces --network-interface-ids "$ENI" --query
'NetworkInterfaces[0].Association.PublicIp' --output text)
curl "http://${PUBLIC_IP}:8080/hello-lucky/hello?country=US"
```

> The manual task definition has an execution role only. After the app started reading Parameter Store and Secrets Manager, a second role (the task role) became necessary. That role is in the Terraform deploy below.

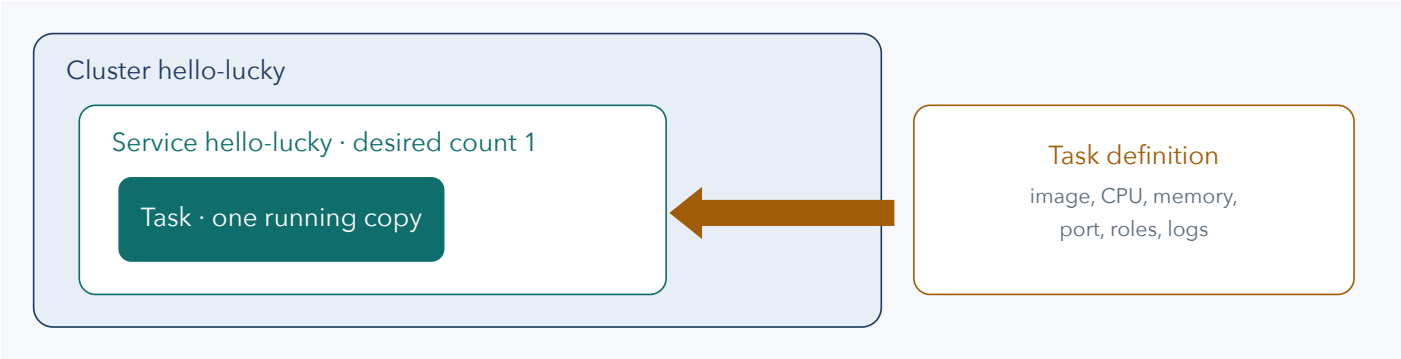


Figure 4. The task definition is the recipe. The service, inside the cluster, keeps one task running from that recipe.

2. Run using Terraform

The files in `terraform/` describe the same infrastructure plus the Phase 3 data. `terraform apply` creates resources in dependency order. A subnet uses `aws_vpc.hello_lucky.id` instead of a pasted `$VPC_ID`. `terraform.tfstate` remembers the real ids. `terraform destroy` removes the stack.

File	What it creates
`versions.tf`	AWS provider 5.x, profile `personal`, region `us-east-1`
`network.tf`	VPC, internet gateway, subnets A and B, route table, default route, both associations, security group
`iam.tf`	`hello-lucky-execution` with the ECS execution policy, and `hello-lucky-task` allowed to read the parameter and the secret
`ecs.tf`	ECR repository, log group, cluster, task definition, service
`config.tf`	Parameter `/hello-lucky/phone-country-codes` = `{"US":1,"INDIA":91}` and secret `hello-lucky/lucky-numbers` = `{"US":11,"INDIA":22}`

Order that was used

1. From `terraform/`, `terraform init` once, then `terraform apply` while `ecs.tf` contained only the repository, log group, and cluster. That created 16 resources and no running task.
2. Build and push the image, using the repository URL AWS returned. Terraform also stores it as `aws_ecr_repository.hello_lucky.repository_url`.

   ```
   ./gradlew bootJar
   docker build --platform linux/arm64 -t hello-lucky:latest .
   ACCOUNT=699558695655
   REPO="${ACCOUNT}.dkr.ecr.us-east-1.amazonaws.com/hello-lucky"
   aws ecr get-login-password --region us-east-1 --profile personal \
     | docker login --username AWS --password-stdin "${ACCOUNT}.dkr.ecr.us-east-
   1.amazonaws.com"
   docker tag hello-lucky:latest "${REPO}:latest"
   docker push "${REPO}:latest"
   ```

3. Add `config.tf` and the task definition plus service in `ecs.tf`. The task definition sets `execution_role_arn` and `task_role_arn`. The service uses both subnets, the security group, `assign_public_ip = true`, `desired_count = 1`, and `wait_for_steady_state = true`.

4. `terraform apply` again. The service creation line does not include the public IP. Look it up with the same `list-tasks` / `describe-tasks` / `describe-network-interfaces` commands, then curl port 8080.

> A successful call looks like `{"country":"US","phone-country-code":1,"lucky-number":11}`. Fargate bills while the desired count stays at 1. A standard Parameter Store parameter is free. The secret is about $0.40 per month, plus $0.05 per 10,000 `GetSecretValue` calls.

Remove it

```
terraform destroy
```

Secrets Manager can leave the secret in a recovery window after destroy, and that window still bills. If the name is still listed, finish the delete with:

```
aws secretsmanager delete-secret --secret-id hello-lucky/lucky-numbers \
  --force-delete-without-recovery --profile personal --region us-east-1
```